Πετράκης Ραφαήλ Ιωάννης ⋆ csd5748

ΗΥ-380: $3^{η}$ Σειρά Ασκήσεων

- **Άσκηση $1^{η}$:**

∗ Στην καλύτερη περίπτωση, η "QuickSort" χωρίζει το πρόβλημα σε δυο ίσα υποπροβλήματα σε κάθε αναδρομική κλήση, άρα:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n$$

∗ Έτσι, με βάση το θεώρημα "Master" (και συγκεκριμένα την δεύτερη περίπτωσή του), έχουμε:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n = f(n) \implies T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n \log n)$$

∗ Οπότε, αφού αποδείξαμε ότι η καλύτερη περίπτωση έχει πολυπλοκότητα $\Theta(n \log n)$, συνεπάγεται μαθηματικά ότι ο χρόνος εκτέλεσής της φράσσεται και προς τα κάτω από αυτό το όριο. Δηλαδή είναι:

$$\boxed{\Omega(n \log n)}$$

- **Άσκηση $2^{η}$:**

∗ Αρχικά, γνωρίζουμε τον αλγόριθμο "LCS" (διάλεξη 7), ο οποίος επιστρέφει το μήκος της μεγαλύτερης κοινής υπο-ακολουθίας:

⋆ Algorithm LCS (X , Y):
  For $i = 1$ to $n - 1$:
    L[i,-1] = 0
  For $j = 0$ to $m - 1$:
    L[-1,j] = 0
  For $i = 0$ to $n - 1$:
    For $j = 0$ to $m - 1$:
      If $X_i = Y_j$:
        L[i,j] = L[i-1,j-1] + 1
      Else:
        L[i,j] = max{ L[i-1,j], L[i,j-1] }
  Return L

∗ Έτσι, μπορούμε να δημιουργήσουμε τον ακόλουθο αλγόριθμο (για την άσκηση):

⋆ Algorithm CheckSubsequence (S1, S2):

n = length(S1)

m = length(S2)

min_length = min(n, m)

matrix = LCS(S1, S2) \\ Καλούμε τον παραπάνω αλγόριθμο ...

lcs_length = matrix[n-1, m-1] \\ Το αποτέλεσμα είναι κάτω-δεξιά στον πίνακα ...

\\ Ελέγχουμε αν το ένα είναι μέσα στο άλλο ή όχι ...

if (lcs_length == min_length): return True;

else: return False;

• **Άσκηση 3η:**

α) Πολυωνυμικός: καλός. Εκθετικός: κακός. ⟶ Αληθής & Ψευδής.

∗ Γενικά, οι αλγόριθμοι πολυωνυμικού χρόνου θεωρούνται ¨αποδοτικοί¨, ενώ οι εκθετικοί θεωρούνται ¨μη αποδοτικοί¨. Όμως, εάν δεν γνωρίζουμε απολύτος τίποτα για το x, δεν μπορούμε να απαντήσουμε με σιγουριά.

β) Radix sort δουλεύει σωστά αν χρησιμοποιείς οποιονδήποτε σωστό αλγόριθμο ταξινόμησης για κάθε ψηφίο. ⟶ Ψευδής.

∗ Ο Radix Sort προϋποθέτει τον αλγόριθμος που χρησιμοποιείται για ταξινόμηση του κάθε ψηφίου είναι αυστηρά σταθερός/ευσταθής.

γ) Δοθέντος ενός πίνακα $A[1 :: n]$ από ακεραίους, ο χρόνος που παίρνει ο Counting Sort είναι πολυωνυμικός σε σχέση με το μέγεθος εισόδου n. ⟶ Ψευδής.

∗ Η χρονική πολυπλοκότητα του Counting Sort είναι $O(n+k)$, όπου k είναι η διαφορά μεταξύ της μέγιστης και της ελάχιστης τιμής στον πίνακα. Αν οι ακέραιοι έχουν τεράστιες τιμές, τότε ο χρόνος εκτέλεσης γίνεται εκθετικός ως προς n, όχι πολυωνυμικός.

δ) Δοθέντος ενός πίνακα $A[1 :: n]$ από ακεραίους, ο χρόνος που παίρνει ο HeapSort είναι πολυωνυμικός σε σχέση με το μέγεθος εισόδου n. ⟶ Αληθής.

∗ Ο αλγόριθμος HeapSort έχει χρονική πολυπλοκότητα $O(n \log n)$ στη χειρότερη περίπτωση. Αφού το $n \log n$ φράσσεται από πάνω με πολυώνυμο, ανήκει στον πολυωνυμικό χρόνο.

ε) Για έναν αλγόριθμο Δυναμικού Προγραμματισμού, ο υπολογισμός όλων των τιμών με bottom-up είναι ασυμπτωτικά ταχύτερος από την χρησιμοποίηση αναδρομής και memoization. ⟶ Ψευδής.

∗ Ασυμπτωτικά (σε συμβολισμό Big-O), και οι δύο προσεγγίσεις έχουν τον ίδιο χρόνο. Το bottom-up απλά γλιτώνει το σταθερό υπολογιστικό κόστος των αναδρομικών κλήσεων. Υπάρχουν περιπτώσεις που το memoization μπορεί να είναι ταχύτερο.

ζ) Ο χρόνος ενός δυναμικού αλγορίθμου είναι πάντα $O(P)$ όπου P είναι ο αριθμός των υποπροβλημάτων. ⟶ Ψευδής.

∗ Ο συνολικός χρόνος προκύπτει πολλαπλασιάζοντας τον αριθμό των υποπροβλημάτων (P) με τον χρόνο που χρειάζεται για να λυθεί το κάθε υποπρόβλημα. Αν η μετάβαση απαιτεί μη-σταθερό χρόνο $O(t)$, τότε η συνολική πολυπλοκότητα είναι $O(P \cdot t)$, όχι απλά $O(P)$.

η) Κάθε πρόβλημα που ανήκει στο NP επιλύεται σε εκθετικό χρόνο. ⟶ Αληθής.

∗ Στην κλάση NP ανήκουν τα προβλήματα των οποίων η λύση μπορεί να επαληθευτεί σε πολυωνυμικό χρόνο. Μπορούμε πάντα να δημιουργήσουμε έναν απλό ντετερμινιστικό αλγόριθμο ο οποίος θα δοκιμάζει εξαντλητικά (brute-force) όλες τις πιθανές λύσεις, διαδικασία που απαιτεί εκθετικό χρόνο.

...

• **Άσκηση 4η:**

α. Ιχνηλάτηση διαδικασίας ΔΙΑΜΕΡΙΣΗ στη συστοιχία: A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11] ...

1. A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]
2. $p = 1$ & $r = 12$ - 1
3. $x = \text{A}[12] = 11$
4. $i = p - 1 = 0$
5.

i	j	Συνθήκη	Ενέργεια	Νέο i	Πίνακας A μετά την επανάληψη
1	13	Ψευδής	Καμία	0	$\langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$
2	19	Ψευδής	Καμία	0	$\langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$
3	9	Αληθής	Αλλαγή $A[1] \leftrightarrow A[3]$	1	$\langle 9, 19, 13, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$
4	5	Αληθής	Αλλαγή $A[2] \leftrightarrow A[4]$	2	$\langle 9, 5, 13, 19, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$
5	12	Ψευδής	Καμία	2	$\langle 9, 5, 13, 19, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$
6	8	Αληθής	Αλλαγή $A[3] \leftrightarrow A[6]$	3	$\langle 9, 5, 8, 19, 12, 13, 7, 4, 21, 2, 6, 11 \rangle$
7	7	Αληθής	Αλλαγή $A[4] \leftrightarrow A[7]$	4	$\langle 9, 5, 8, 7, 12, 13, 19, 4, 21, 2, 6, 11 \rangle$
8	4	Αληθής	Αλλαγή $A[5] \leftrightarrow A[8]$	5	$\langle 9, 5, 8, 7, 4, 13, 19, 12, 21, 2, 6, 11 \rangle$
9	21	Ψευδής	Καμία	5	$\langle 9, 5, 8, 7, 4, 13, 19, 12, 21, 2, 6, 11 \rangle$
10	2	Αληθής	Αλλαγή $A[6] \leftrightarrow A[10]$	6	$\langle 9, 5, 8, 7, 4, 2, 19, 12, 21, 13, 6, 11 \rangle$
11	6	Αληθής	Αλλαγή $A[7] \leftrightarrow A[11]$	7	$\langle 9, 5, 8, 7, 4, 2, 6, 12, 21, 13, 19, 11 \rangle$

6. $A[i+1] = A[8] = 12 \leftrightarrow A[r] = A[11] = 11 \implies < 9, 5, 8, 7, 4, 2, 6, 11, 21, 13, 19, 12 >$
7. Return i + 1 = 8;

βi. Η ΔΙΑΜΈΡΙΣΗ αν όλα τα στοιχεία της συστοιχίας έχουν την ίδια τιμή επιστρέφει: $q = i + 1 = r$ ...

βii. Τροποποιημένη ΔΙΑΜΕΡΙΣΗ ώστε να επιστρέφει: $q = \lfloor (p + r)/2 \rfloor$:

⋆ Algorithm PARTITION(A, p, r):

```
// Έλεγχος αν όλα τα στοιχεία είναι ίσα:
all_equal ← true
for k ← p to r - 1
  if A[k] ≠ A[r] then
      all_equal ← false
       break
if all_equal then
  return ⌊(p + r)/2⌋
// Συνεχίζουμε κανονικά:
x ← A[r]
i ← p - 1
for j ← p to r - 1
  if A[j] ≤ x then
      i ← i + 1
      swap A[i] ↔ A[j]
swap A[i + 1] ↔ A[r]
return i + 1
```

γ. Χρονική πολυπλοκότητα είναι Θ(n) αφού έχουμε:

1. Αρχικοποιήσεις $\longrightarrow$ $\Theta(1)$
2. Βρόχος $\longrightarrow$ $\Theta(n)$ αφού είναι $r - 1 - p + 1 = r - p$ επαναλήψεις σε πίνακα μεγέθους $n = r - p + 1$, άρα $n - 1$ φορές συνολικά ...
3. Λειτουργίες εντός και μετά του βρόχου $\longrightarrow$ $\Theta(1)$ (συγκρίσεις, προσθέσεις, αντιμεταθέσεις, επιστροφή, κλπ.) ...

• **Άσκηση 5η:**

α. Η διαδικασία της ΤΑΧΥΤΑΞΙΝΟΜΗΣΗ (QUICK SORT) καλεί εκείνη της ΔΙΑΜΕΡΙΣΗ (PARTITION), σύμφωνα με την εκφώνηση. Πρέπει να τροποποιήσουμε την πρώτη ώστε να ταξινομεί τα στοιχεία σε μη αύξουσα σειρά αντί για φθίνουσα. Εάν επιτρέπεται (δεν είναι σαφές) να επεξεργαστούμε την διαδικασία την ΔΙΑΜΕΡΙΣΗ (PARTITION), απλά αλλάζει η τέταρτη γραμμή της από "$if\ \ A[j] <= x\ \ then$" σε "$if\ \ A[j] >= x\ \ then$". Αν, όμως, πρέπει να τροποποιηθεί η -κανονική- ΤΑΧΥΤΑΞΙΝΟΜΗΣΗ (ΧΥΓΚ ΣΟΡΤ), τότε απλά προσθέτουμε στο τέλος της κώδικα ο οποίος αντιστρέφει τον πίνακα Α ("REVERSE(A)") και έτσι θα μετατραπεί σε αύξουσα σειρά ...

β. Όταν όλα τα στοιχεία της συστοιχίας Α έχουν την ίδια τιμή, τότε ο χρόνος εκτέλεσης της διαδικασίας ΤΑΧΥΤΑΞΙΝΟΜΗΣΗ (QUICK SORT) είναι $\Theta(n^2)$, αφού έχουμε:

1. 1 σύγκριση $\longrightarrow\ \Theta(1)$
2. PARTITION() $\longrightarrow\ \Theta(n)$ (από προηγούμενη άσκηση και ερώτημα γ)
3. Αναδρομές $\longrightarrow\ \Theta(n)$ επειδή το βάθος των αναδρομών θα είναι n (ελέγχονται όλα τα στοιχεία τον άνισων κομματιών) ...

γ. Όταν η συστοιχία Α έχει διαφορετικά στοιχεία αλλά είναι ήδη ταξινομημένη κατά φθίνουσα σειρά, τοτε ο χρόνος εκτέλεσης της ΤΑΧΥΤΑΞΙΝΟΜΗΣΗΣ είναι $\Theta(n^2)$, αφού:

1. 1 σύγκριση $\longrightarrow\ \Theta(1)$
2. PARTITION() $\longrightarrow\ \Theta(n)$ (οδηγός $= A[r] =$ ελάχιστο στοιχείο, αρα $A[j] \leq x$ πάντα ψευδής και η διαδικασία επιστρέφει $q = p$)
3. Αναδρομές $\longrightarrow\ \Theta(n)$ επειδή βάθος αναδρομών n (ο πίνακας χωρίζεται σε δύο κομμάτια μεγέθους 0 και $n-1$ σε κάθε βήμα) ...

δ. Χρόνος καλύτερης περίπτωσης ΤΑΧΥΤΑΞΙΝΟΜΗΣΗΣ σε συστοιχία με διαφορετικά ανά δύο στοιχεία, είναι: $O(nlgn)$ - επειδή:

∗ Όταν όλα τα στοιχεία είναι διαφορετικά μεταξύ τους, ο πίνακας θα χωρίζεται ακριβώς στην μέση σε κάθε βήμα, οπότε ισχύει:

$$T(n) = 2T(n/2) + \Theta(n) \quad [\Theta(n) \rightarrow \text{PARTITION()}]$$

∗ Άρα, σύμφωνα με την δεύτερη περίπτωση του θεωρήματος μάστερ, η πολυπλοκότητα θα είναι: $O(nlgn)$ ...

• **Άσκηση 6η:**

⋆. Δεν μας δίνεται ο ψευδοκώδικας για την διαδικασία ΑΠΑΡΙΘΜΗΤΙΚΗ ΤΑΞΙΝΟΜΗΣΗ. Έστω:

⋆ Algorithm COUNTING-SORT (A, B, n, k):

```
// Let: C[0..k]: temp, [0..n]: input, B[0..n]: output, n = len(A), k = max(A) ...
for i = 0 to k:
    C[i] = 0
for j = 1 to n:
    C[A[j]] = C[A[j]] + 1
for i = 1 to k:
    C[i] = C[i] + C[i - 1]
for j = n downto 1:
    B[C[A[j]]] = A[j]
    C[A[j]] = C[A[j]] - 1
return B;
```

∗ Περιγραφή: "A = < 6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 >" ...

0. A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2], $n = 11$, $k = 6$, $C[k+1]$; και B[n];

1. $C = [\,0, 0, 0, 0, 0, 0, 0\,]$; // Αρχικοποίηση ...

2. $C = [\,2, 2, 2, 2, 1, 0, 2\,]$; // Πίνακας Συχνοτήτων ...

3. $C = [\,2, 4, 6, 8, 9, 9, 11\,]$; // Κάθε θέση δείχνει το άθροισμα των προηγούμενων δύο κελιών ...

4.

Βήμα (j)	Στοιχείο (A[j])	Ανάγνωση θέσης (C[A[j]])	Τοποθέτηση στον πίνακα B	Ενημέρωση πίνακα C
11	2	C[2]=6	B[6]=2	C[2]=6-1→5
10	3	C[3]=8	B[8]=3	C[3]=8-1→7
9	1	C[1]=4	B[4]=1	C[1]=4-1→3
8	6	C[6]=11	B[11]=6	C[6]=11-1→10
7	4	C[4]=9	B[9]=4	C[4]=9-1→8
6	3	C[3]=7	B[7]=3	C[3]=7-1→6
5	1	C[1]=3	B[3]=1	C[1]=3-1→2
4	0	C[0]=2	B[2]=0	C[0]=2-1→1
3	2	C[2]=5	B[5]=2	C[2]=5-1→4
2	0	C[0]=1	B[1]=0	C[0]=1-1→0
1	6	C[6]=10	B[10]=6	C[6]=10-1→9

5. Ο τελικός ταξινομημένος πίνακας θα είναι: $B = [\,0, 0, 1, 1, 2, 2, 3, 3, 4, 6, 6\,]$ ...

• **Άσκηση 7η:**

⋆ Η Αριθμοτακτική Ταξινόμηση (Radix Sort) για λέξεις λειτουργεί ταξινομώντας τα στοιχεία γράμμα προς γράμμα, ξεκινώντας από το τελευταίο γράμμα (δεξιά) προς το πρώτο (αριστερά). Έτσι, έχουμε την περιγραφή ως εξής:

1. Αρχική Λίστα: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX

// 1ο Πέρασμα: Εκτέλεση με κλειδί τον 3ο χαρακτήρα ($i = 3$) και N=26 κάδους ...

2. Νέα Λίστα: SEA, TEA, MOB, TAB, DOG, RUG, DIG, BIG, BAR, EAR, TAR, COW, ROW, NOW, BOX, FOX

// Ταξινόμηση ως προς το 2ο (μεσαίο) γράμμα ...

3. Νεα Λίστα: TAB, BAR, EAR, TAR, SEA, TEA, DIG, BIG, MOB, DOG, COW, ROW, NOW, BOX, FOX, RUG

// Ταξινόμηση ως προς το 1ο (πρώτο) γράμμα ...

4. Τελική Λίστα: BAR, BIG, BOX, COW, DIG, DOG, EAR, FOX, MOB, NOW, ROW, RUG, SEA, TAB, TAR, TEA

• **Άσκηση 8η:**

⋆ Η διαδικασίας ΤΑΞΙΝΟΜΗΣΗ ΜΕ ΔΟΧΕΙΑ (BUCKET SORT) στη συστοιχία A = < 0.75, 0.13, 0.16, 0.64, 0.39, 0.20, 0.89, 0.53, 0.71, 0.42, 0.19 >, χωρίζοντας το διάστημα [0, 1) σε 10 ίσου μεγέθους υποδιαστήματα:

1. Αρχικοποίηση Δοχείων: Δημιουργούμε 10 κενά δοχεία $B[0]$ έως $B[9]$: B[0] τιμές 0.00 - 0.09, ..., B[9] τιμές 0.90 - 0.99.

2. Κατανομή στοιχείων:

Στοιχείο (e)	Υπολογισμός Δείκτη: $k = \lfloor 10 \times e \rfloor$	Δοχείο Προορισμού	Περιεχόμενο Δοχείου (τελικά)
0.75	$7.5 \to 7$	B[7]	B[7]: [0.75, 0.71]
0.13	$1.3 \to 1$	B[1]	B[1]: [0.13, 0.16, 0.19]
0.16	$1.6 \to 1$	B[1]	B[1]: [0.13, 0.16, 0.19]
0.64	$6.4 \to 6$	B[6]	B[6]: [0.64]
0.39	$3.9 \to 3$	B[3]	B[3]: [0.39]
0.20	$2.0 \to 2$	B[2]	B[2]: [0.20]
0.89	$8.9 \to 8$	B[8]	B[8]: [0.89]
0.53	$5.3 \to 5$	B[5]	B[5]: [0.53]
0.71	$7.1 \to 7$	B[7]	B[7]: [0.75, 0.71]
0.42	$4.2 \to 4$	B[4]	B[4]: [0.42]
0.19	$1.9 \to 1$	B[1]	B[1]: [0.13, 0.16, 0.19]

3. Ταξινόμηση: Μόνο το δοχείο "$B[7] : [0.75, 0.71]$" θα γίνει "$B[7] : [0.71, 0.75]$" ...

4. Συγχώνευση: Τελικός Πίνακας $\underline{\text{A} = < 0.13, 0.16, 0.19, 0.20, 0.39, 0.42, 0.53, 0.64, 0.71, 0.75, 0.89 >}$ ...

• **Άσκηση 9η:**

⋆ Οι αλγόριθμοι "Insertion Sort" (Ενθετική Ταξινόμηση) και "Merge Sort" (Συγχωνευτική Ταξινόμηση) είναι (η μπορούν να υλοποιηθούν ως) ευσταθείς (stable), ενώ οι αλγόριθμοι "Heap Sort" (Ταξινόμηση Σορού) και "Quick Sort" (Ταχυταξινόμηση) όχι ...

• **Άσκηση 10η:**

αi. Περιγραφή του Αλγορίθμου: Για να δώσουμε ρέστα από $\{25, 10, 5, 1\}$ σεντς για (ακέραιο) n σεντς, επιλέγουμε επανειλημμένα το νόμισμα με τη μεγαλύτερη δυνατή αξία που δεν υπερβαίνει το υπολειπόμενο ποσό. Αφαιρούμε την αξία του νομίσματος από το υπολειπόμενο ποσό και επαναλαμβάνουμε τη διαδικασία μέχρι το ποσό να γίνει 0.

αii. Απόδειξη Βελτιστότητας: Η βασική ιδέα είναι πως κανένας συνδυασμός μικρότερων κερμάτων δεν αντικαθιστά καλύτερα μεγαλύτερο. Για παράδειγμα: 1×25 είναι καλύτερη επιλογή από τα $2 \times 10 + 1 \times 5$, $2 \times 10 + 5 \times 1$, $1 \times 10 + 3 \times 5$, $1 \times 10 + 2 \times 5 + 5 \times 1$, $1 \times 10 + 1 \times 5 + 10 \times 1$, κλπ. ...

β. ???

γ. Έστω ότι έχουμε το σύνολο νομισμάτων: $\{8, 5, 1\}$ και θέλουμε να δώσουμε ρέστα για το ποσό: $n = 10$. Η λύση του Άπληστου Αλγορίθμου θα ήταν: 1 νόμισμα του 8 και 2 νομίσματα του 1, σύνολο 10 με 3 κέρματα. Όμως, η βέλτιστη λύση θα ήταν: 2 νομίσματα του 5, σύνολο 10 με 2 κέρματα.

δ. Ψευδοκώδικας:

∗ Η κεντρική ιδέα του αλγορίθμου είναι ότι βρίσκει τη βέλτιστη λύση για το τελικό ποσό χτίζοντάς την βήμα-βήμα: δοκιμάζει κάθε διαθέσιμο νόμισμα και προσθέτει 1 στην ήδη υπολογισμένη βέλτιστη λύση του υπολειπόμενου ποσού, επιλέγοντας τελικά αυτό που δίνει τα λιγότερα συνολικά νομίσματα. Τέλος, η πολυπλοκότητα είναι όντως: $T(n) = O(nk)$, αφού η πρώτη επανάληψη είναι n φορές για O(1), η δεύτερη είναι n φορές για την τρίτη, η οποία είναι k φορές για O(1), οπότε $O(nk)$ ...

⋆ Algorithm MINCOINS(coins, n):

```
// Είσοδος: πίνακας k θέσεων με αξίες νομισμάτων από 1 και το συνολικό ποσό για το οποίο θέλουμε ρέστα ...
mc[0] ← 0
for i ← 1 to n:
  mc[i] ← +∞
for i ← 1 to n:
  for every j in coins:
      if j <= i:
          if 1 + mc[i - j] < mc[i]:
              mc[i] = 1 + mc[i - j]
return mc[n];
```


LaTeX